

Natural Language Processing

3. Language models

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Language Models
- Estimating N-gram Probabilities
- Evaluating Language Models
- Smoothing

Language Models



Language Models

- Goal: to define the probability of a word in context

$$P(w|c)$$

- Seen a possible use in spelling correction:

$$\hat{w} = \operatorname{argmax}_{w \in C} \underbrace{P(x|w)}_{\text{channel model}} \underbrace{P(w|c)}_{\text{language model}}$$

- In fact, a fundamental concept in NLP
- Intuitive uses in speech recognition, Augmentative and Alternative Communication Systems, natural language generation, text classification, ...



Probabilistic Language Models

- The context is generally a sequence of words/tokens

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- Another way of putting it: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \dots, w_n)$$

- A model that computes $P(W)$ or $P(w_5 | w_1, w_2, w_3, w_4)$ is called a **language model**



Reminder: The Chain Rule

- **Notation:** w_1^n to indicate a sequence of n words $w_1, w_2, \dots, w_n$
- Chain Rule:

$$\begin{aligned}P(w_1^n) &= P(w_1)P(w_2|w_1)P(w_3|w_1^2)\dots P(w_n|w_1^{n-1}) \\ &= \prod_{i=1}^n P(w_i|w_1^{i-1})\end{aligned}$$

- $P(\text{"its water is so transparent"}) = P(\text{its}) P(\text{water}|\text{its}) P(\text{is}|\text{its water})$
 $P(\text{so}|\text{its water is}) P(\text{transparent}|\text{its water is so})$



How to Estimate these Probabilities

- Could we just count and divide?

$$P(\textit{the} | \textit{its water is so transparent that}) = \frac{\textit{Count}(\textit{its water is so transparent that the})}{\textit{Count}(\textit{its water is so transparent that})}$$

- No! too many possible sentences
- We'll never see enough data for estimating these



Markov Assumption

- Simplifying assumption:

$$P(\text{the}|\text{its water is so transparent that}) \approx P(\text{the}|\underbrace{\text{that}}_{k=1})$$

- or maybe:

$$P(\text{the}|\text{its water is so transparent that}) \approx P(\text{the}|\underbrace{\text{transparent that}}_{k=2})$$

- In general, for $k \geq 1$:

$$P(w_i|w_1^{i-1}) \approx P(w_i|w_{i-k}^{i-1})$$

therefore:

$$P(w_1^n) \approx \prod_{i=1}^n P(w_i|w_{i-k}^{i-1})$$



Unigram Model

- The simplest case is the one where history doesn't matter ($k = 0$)

$$P(w_1^n) \approx \prod_i P(w_i)$$

- What kind of knowledge does this model embody?

Sampling Sentences from a LM

- Sampling can be used to visualize what kind of knowledge an LM embodies

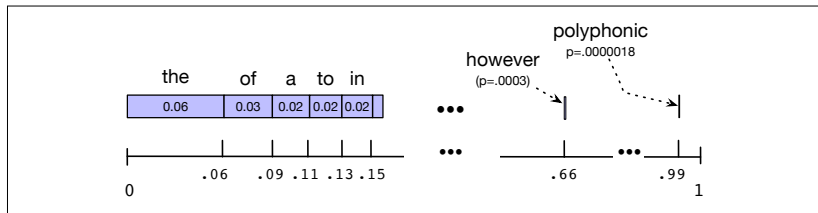


Figure 3.3 A visualization of the sampling distribution for sampling sentences by repeatedly sampling unigrams. The blue bar represents the frequency of each word. The number line shows the cumulative probabilities. If we choose a random number between 0 and 1, it will fall in an interval corresponding to some word. The expectation for the random number to fall in the larger intervals of one of the frequent words (*the*, *of*, *a*) is much higher than in the smaller interval of one of the rare words (*polyphonic*).



Unigram Model

$$P(w_1^n) \approx \prod_i P(w_i)$$

- Some automatically generated sentences from a unigram model:
 - *fifth an of futures the an incorporated a a the inflation most dollars quarter in is mass*
 - *thrift did eighty said hard 'm july bullish*
 - *that or limited the*



Bigram Model

- Only the previous word matters ($k = 1$)

$$P(w_1^n) \approx \prod_i P(w_i | w_{i-1})$$

- Some automatically generated sentences from a bigram model:
 - *texaco rose one in this issue is pursuing growth in a boiler house said mr. gurria mexico 's motion control proposal without permission from five hundred fifty five yen*
 - *outside new car parking lot of the agreement reached*
 - *this would be a record november*



N-Gram Models

- We can extend to trigrams, 4-grams, 5-grams
- In general this is an insufficient model of language
 - **long-range dependencies!**
 - *The computer(s) which I had just put into the machine room on the fifth floor is (are) crashing*
- But we can often get away with N-gram models

Estimating N-gram Probabilities



Estimating Bigram Probabilities

- The **Maximum Likelihood Estimate**

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1} w_i)}{\text{Count}(w_{i-1})}$$

Consider the following corpus:

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$\begin{array}{lll} P(I | \text{<s>}) = 2/3 & P(\text{Sam} | \text{<s>}) = 1/3 & P(\text{am} | I) = 2/3 \\ P(\text{</s>} | \text{Sam}) = ? & P(\text{Sam} | \text{am}) = ? & P(\text{do} | I) = ? \end{array}$$

- What is $P(\text{<s> I am Sam </s>})$?



Another Example: Berkeley Restaurant Project

- A corpus of user queries to a dialogue system about restaurants in Berkeley, California such as the following (text-normalized) ones:

can you tell me about any good cantonese restaurant close by
mid priced thai food is what i'm looking for
tell me about chez panisse
can you give me a listing of the kinds of food that are available
i'm looking for a good place to eat breakfast
when is caffe venezia open during the day

- Some unigram counts:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278



Raw Bigram Counts

- Some bigram **counts** $\{Count(w_{i-1}w_i)\}$

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray.

- To obtain bigram **probabilities**: $P(w_i|w_{i-1}) = \frac{Count(w_{i-1}w_i)}{Count(w_{i-1})}$



Raw Bigram Probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

- Other useful **probabilities**:

$$P(i|<s>) = 0.25$$

$$P(\text{english}|\text{want}) = 0.0011$$

$$P(\text{food}|\text{english}) = 0.5$$

$$P(</s>|\text{food}) = 0.68$$

- What is $P(<s> \text{ I want English food } </s>)$?



What Kinds of Knowledge?

- What linguistic phenomena are captured by this LM?
 - $P(\text{english}|\text{want}) = .0011$
 - $P(\text{chinese}|\text{want}) = .0065$
 - $P(\text{to}|\text{want}) = .66$
 - $P(\text{eat}|\text{to}) = .28$
 - $P(\text{food}|\text{to}) = 0$
 - $P(\text{want}|\text{spend}) = 0$
 - $P(i|<s>) = .25$
- **syntactic**
- personal assistant
- cultural

Evaluating Models



How Good Is Our Model?

- Does our language model prefer good sentences to bad ones?
 - Assign higher probability to “real” or “frequently observed” sentences
 - Than “ungrammatical” or “rarely observed” sentences?
- We train parameters of our model on a **training set**
- We test the model's performance on data we haven't seen
 - A **test set** is an unseen dataset that is different from our training set, totally unused (**held out**)
 - An **evaluation metric** tells us how well our model does on the test set
 - **Don't** peek at the test set 😊
 - Use **devset** (also held out) for hyperparameter tuning
- Evaluation: a model is better if it assigns a **higher probability** to the test set, meaning it more accurately predicts/fits the test set
- Two approaches: extrinsic and intrinsic evaluation



Extrinsic Evaluation

Extrinsic/end-to-end/in-vivo/downstream evaluation

- The only way to know if an improvement in a component is really going to help the task at hand
 - 1 Put each model in a task
 - Spelling corrector, speech recognizer, MT system
 - 2 Run the task, get an accuracy for A and for B
 - How many misspelled words corrected properly
 - How many words recognized or translated correctly
 - 3 Compare accuracy for A and B
- Best evaluation for comparing models A and B
- However, running NLP systems end-to-end is often very costly/time-consuming



Intrinsic Evaluation

Measures quality of component **independently of application**

- **Metric** to evaluate *potential improvements* in a LM
- Probability-based metric: **perplexity**
- Bad approximation
 - unless the test data looks **just** like the training data
 - So **generally only useful in pilot experiments**
 - Improvement in perplexity must be corroborated by downstream evaluation before drawing conclusions
- But is helpful to think about



Intuition of Perplexity

The new method of estimating entropy exploits the fact that anyone speaking a language possesses, implicitly, an enormous knowledge of the statistics of the language. Familiarity with the words, idioms, clichés and grammar enables him to fill in missing or incorrect letters in proof-reading, or to complete an unfinished phrase in conversation. An experimental demonstration of the extent to which English is predictable can be given as follows: Select a short passage unfamiliar to the person who is to do the predicting. He is then asked to guess the first letter in the passage. If the guess is correct he is so informed, and proceeds to guess the second letter. If not, he is told the correct first letter and proceeds to his next guess. This is continued through the text. As the experiment progresses, the subject writes down the correct text up to the current point for use in predicting future letters. The result of a typical experiment of this type is given below. Spaces were included as an additional letter, making a 27 letter alphabet. The first line is the original text; the second line contains a dash for each letter correctly guessed. In the case of incorrect guesses the correct letter is copied in the second line.

```
(1) THE ROOM WAS NOT VERY LIGHT A SMALL OBLONG
(2) ---ROO-----NOT-V-----I-----SM---OBL----      (8)

(1) READING LAMP ON THE DESK SHED GLOW ON
(2) REA-----O-----D---SHED-GLO--O--

(1) POLISHED WOOD BUT LESS ON THE SHABBY RED CARPET
(2) P-L-S-----O---BU--L-S--O-----SH-----RE--C-----
```

C Shannon, *Prediction and Entropy of written English* (1951)



Intuition of Perplexity

- The Shannon Game:
 - How well can we predict the next character/word?
 - I always order pizza with cheese and ...
 - The 33rd President of the US was ...
 - I saw a ...
 - Unigrams are terrible at this game. (Why?)
 - A better model of a text is one which assigns a higher probability to the word that actually occurs



Perplexity

- The best language model is one that best predicts an unseen test set
 - Gives the highest $P(\text{sentence})$
- **Perplexity is the inverse probability of the test set, normalized by the number of words**

$$P(w_1^N) = \prod_i P(w_i | w_1^{i-1}) \stackrel{\text{bigrams}}{\approx} \prod_i P(w_i | w_{i-1})$$

$$PP(w_1^N) = P(w_1^N)^{-\frac{1}{N}} \stackrel{\text{bigrams}}{=} \sqrt[N]{\prod_i \frac{1}{P(w_i | w_{i-1})}}$$



Perplexity as Weighted Average Branching Factor

- Let's suppose a sentence consisting of random digits
- What is the perplexity of this sentence according to a model that assigns $P = 1/10$ to each digit?

$$\begin{aligned} PP(w_1^N) &= P(w_1^N)^{-\frac{1}{N}} \\ &= \left(\underbrace{\frac{1}{10} \times \frac{1}{10} \times \dots \times \frac{1}{10}}_{N \text{ times}} \right)^{-\frac{1}{N}} = \frac{1}{10}^{-1} = 10 \end{aligned}$$

- What if...
 - training set: 0 occurs 91 times, each other digit just once
 - test set: 0 0 0 0 0 3 0 0 0 0
 - perplexity?



Lower Perplexity = Better Model

- WSJ corpus
- Training 38 million words, test 1.5 million words

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- Minimize perplexity = maximize test set probability according to the language model
- Notice: perplexity is measured on a given test set
 - How about other test sets?
 - How *general* are these language models?



Approximating Shakespeaere

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Figure 3.3 Eight sentences randomly generated from four n-grams computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

- $N = 884K$, $|V|=30K$, only 300K bigram types: 0.04% of $|V|^2$
- Quadrigrams: looks like Shakespeare because it *is* Shakespeare



The WSJ Is No Shakespeare

1
gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2
gram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Figure 3.4 Three sentences randomly generated from three n-gram models computed from 40 million words of the *Wall Street Journal*, lower-casing all characters and treating punctuation as words. Output was then hand-corrected for capitalization to improve readability.

- Guessing the author of these random trigram sentences is easy
- N-grams are better at modeling the training corpus as N increases
- N-gram model is dependent on the training corpus



The Perils of Overfitting

- We need to train robust models that **generalize**
- Models work well for prediction if test corpus $\approx$ training corpus
 - At least, same **genre** and **dialect**
- Problem is **sparsity**:
 - We might have a good estimate of probability only for n-grams occurring enough times in training
 - N-grams not seen in the training corpus have **zero probability**!
 - But they may occur in the test set. Example:
 - seen during training:

denied the allegations (5)

denied the speculation (2)

denied the rumors (1)

denied the report (1)

- seen only during testing:

denied the offer

denied the loan

- What happens to **perplexity**?



Unknown Words

- **Closed vocabulary** systems: we know all words that can occur
 - speech recognition: pronunciation dictionary
 - statistical machine translation: phrase dictionary
- **Open vocabulary** systems have to deal with **unknown** words or **Out-Of-Vocabulary (OOV)** words
 - we add a pseudo-word called <UNK>
- Solution: turn the problem into closed vocabulary
 - either by choosing a vocabulary and modeling all other words as <UNK>
 - or by modeling all least frequent words as <UNK>
- *Warning*: perplexity artificially reduced by choosing small vocabulary
 - Perplexities to be compared only across LM with same vocabulary

Smoothing



Smoothing

- Must keep language model from assigning 0 probability to unseen words or contexts
- Idea: shave off a bit of probability mass from some more frequent words and give it to unseen events
- Smoothing or discounting
- Various smoothing methods
 - Laplace (add-1)
 - Add-k
 - Backoff
 - Interpolation
 - Kneser-Ney
 - Stupid Backoff



Laplace Smoothing

- 1 Add one to all counts
 - 2 Renormalize
- Example: bigram probability estimates

$$\text{MLE} \quad P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}w_i)}{\text{Count}(w_{i-1})}$$

$$\text{Add-1} \quad P_{\text{Laplace}}^*(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}w_i) + 1}{\text{Count}(w_{i-1}) + V}$$

- Adjusted counts: $\text{Count}^*(w_i) = (\text{Count}(w_i) + 1) \frac{N}{N + V}$



Laplace Smoothing

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Figure 3.5 Add-one smoothed bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Previously-zero counts are in gray.

- A significant share of probability mass is moved to all the zeros
- Dramatic impact on counts and probabilities



Add-k Smoothing

- Simple extension: reduce the probability mass removed from seen events

$$P_{Addk}^*(w_i|w_{i-1}) = \frac{Count(w_{i-1}w_i) + k}{Count(w_{i-1}) + kV}$$

- $k = .5? \dots .05? \dots .01?$ need to optimize on a **devset**
- Useful for text classification
- Doesn't work so well for language modeling



Backoff

- Say we need to compute $P(w_i | w_{i-2} w_{i-1})$ but have no examples of a particular trigram $w_{i-2} w_{i-1} w_i$
 - We can estimate the probability using $P(w_i | w_{i-1})$
 - In the worst case, we can look to the unigram $P(w_i)$
- Using **less context** sometimes is a good thing (helps generalize)
- We “*back off*” to lower-order n-gram
- We need additional parameters (α, P^*) to **discount** the higher-order n-grams and save some probability mass

$$P_{\text{Backoff}}(w_i | w_{i-n+1}^{i-1}) = \begin{cases} P^*(w_i | w_{i-n+1}^{i-1}), & \text{if } \text{Count}(w_{i-n+1}^i) > 0 \\ \alpha(w_{i-n+1}^{i-1}) P_{\text{Backoff}}(w_i | w_{i-n+2}^{i-1}), & \text{otherwise} \end{cases}$$



Interpolation

- Mix probability estimates from all n-gram estimators

$$\begin{aligned}\hat{P}(w_i | w_{i-2} w_{i-1}) = & \lambda_1 P(w_i | w_{i-2} w_{i-1}) \\ & + \lambda_2 P(w_i | w_{i-1}) \\ & + \lambda_3 P(w_i)\end{aligned}$$

$$\sum_i \lambda_i = 1$$



Kneser-Ney

- Most commonly used method (among best performing)
- Uses **absolute discounting**
 - Idea: look at a held-out set to check counts for all n-grams that have a given count in training set

Bigram count in training set	Bigram count in heldout set
0	0.0000270
1	0.448
2	1.25
3	2.24
4	3.23
5	4.21
6	5.23
7	6.21
8	7.21
9	8.26

Figure 3.8 For all bigrams in 22 million words of AP newswire of count 0, 1, 2,...,9, the counts of these bigrams in a held-out corpus also of 22 million words.



The Web and Stupid Backoff

- Web → extremely large language models
- 2006: Google releases *n*-grams appearing in $N \geq 40$ distinct books from 1TB+ corpus
 - includes 1B+ five-word sequences using 13M+ unique word types
 - requires special “tricks” for taming large sets
 - probabilities quantized in 4-8 bits
 - *n*-grams stored using efficient tree structures (reverse tries)
- A simpler method than full KN smoothing seems effective
- **Stupid backoff**: backoff without normalization
 - no longer a probability distribution

$$S(w_i | w_{i-k+1}^{i-1}) = \begin{cases} \frac{\text{Count}(w_{i-k+1}^i)}{\text{Count}(w_{i-k+1}^{i-1})} & \text{if } \text{Count}(w_{i-k-1}^i) > 0 \\ \lambda S(w_i | w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

Questions?